ANN using Hierarchical Navigable Small World

Prof. Anirban Dasgupta

Divisht | Aditya Kumar | Yash Patel | Harshita Singh

CS_328 : Intro to Data Science

Indian Institute of Technology, Gandhinagar

21/4/2026

Project Overview

Goal : To implement Approximate Nearest Neighbours using Hierarchical Navigable Small world method using lib : hnswlib

Code and Deployment : Github | Website hosted on Streamlit

Methodology : We have compared Baseline metrics and hypertuning strategies like grid, random and bayesian search for different parameters to reduce the latency of approximate search and get a decent recall for faster search.

We have run the tests on a synthetic dataset using these methods and a sampled dataset to obtain the recall - latency values and have plotted the graphs to then demonstrate the tradeoff.

Approximate Nearest Neighbour

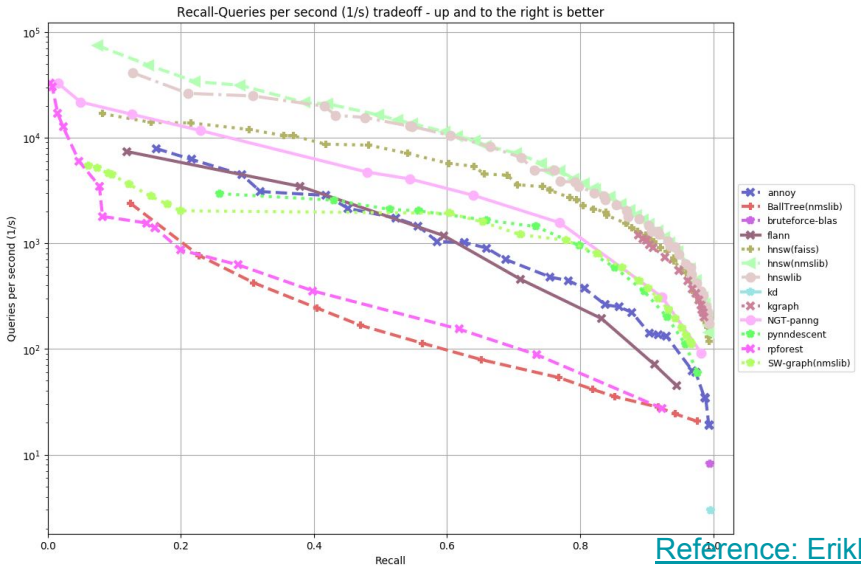
Approximate nearest neighbor (ANN) is an algorithm that finds a data point in a data set that's very close to the given query point, but not necessarily the absolute closest one.

Need to optimise for speed , KNN takes O(N) which is not ideal

Objectives

We need to optimise for both latency as well as Recall in our code which is represented by a Pareto curve. This recall-latency Pareto frontier represents the optimal tradeoff where increasing search accuracy (recall) necessitates higher query latency, forming a concave curve.

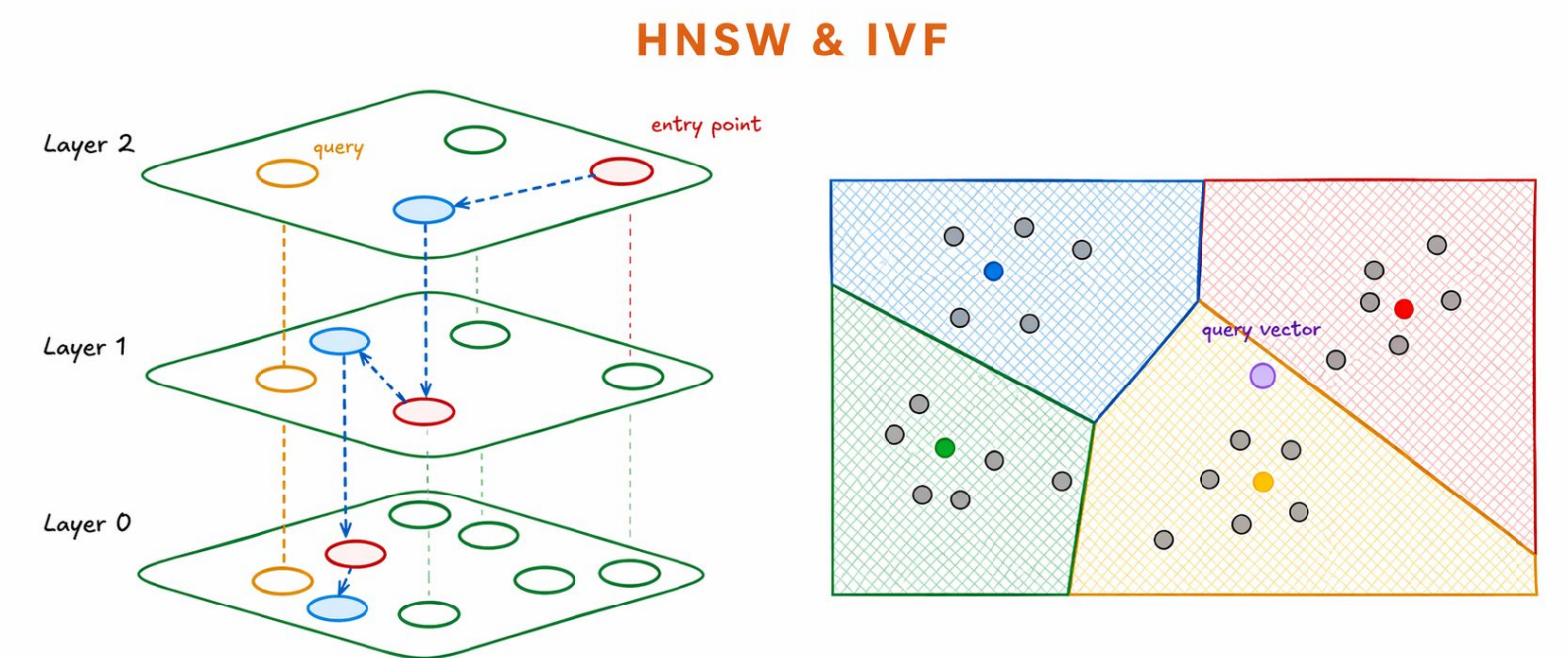
We will look for the concave region for Higher QPS (queries per sec) and high Recall - low latency values.



Reference: Erikbern

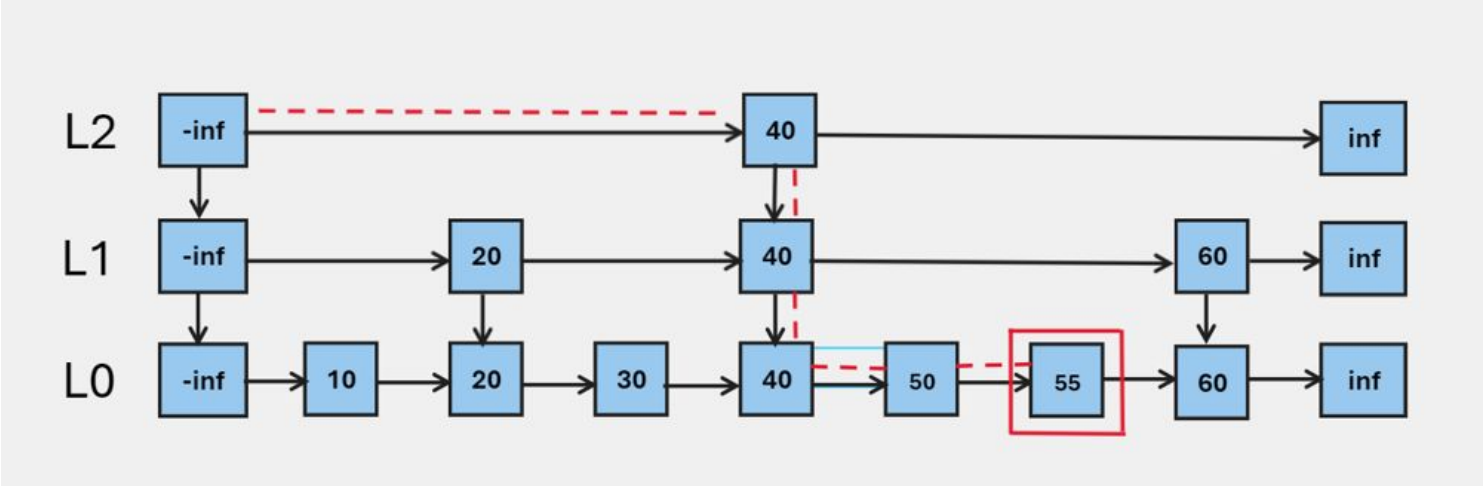
Popular ANN search methods

- The two popular methods - IVF (inverted file indexing) and HNSW.



Motivation behind HNSW - Skip Lists

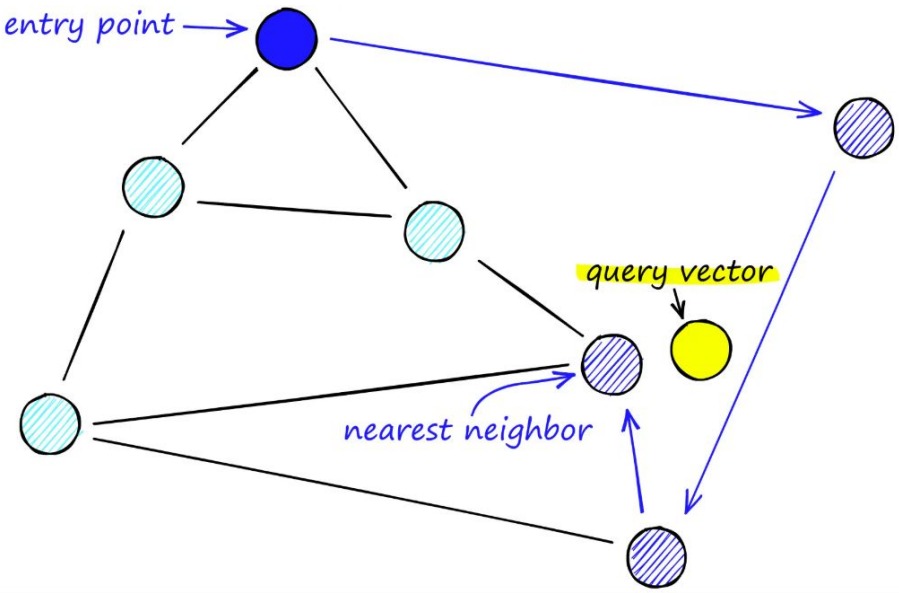
A skip list is a probabilistic, sorted data structure that allows for faster search operations by using multiple layers of linked lists arranged randomly from some probability distribution.



Navigable Small World

NSW (Navigable small world) is the technique for just one layer and is the base idea behind our HNSW implementation.

It works using greedy algorithm - for a starting node , look for the adjacent nodes and calculate the query distance to reach the minimum distance vertex, stop if there is no point closer to the query point than the vertex.



HNSW Implementation

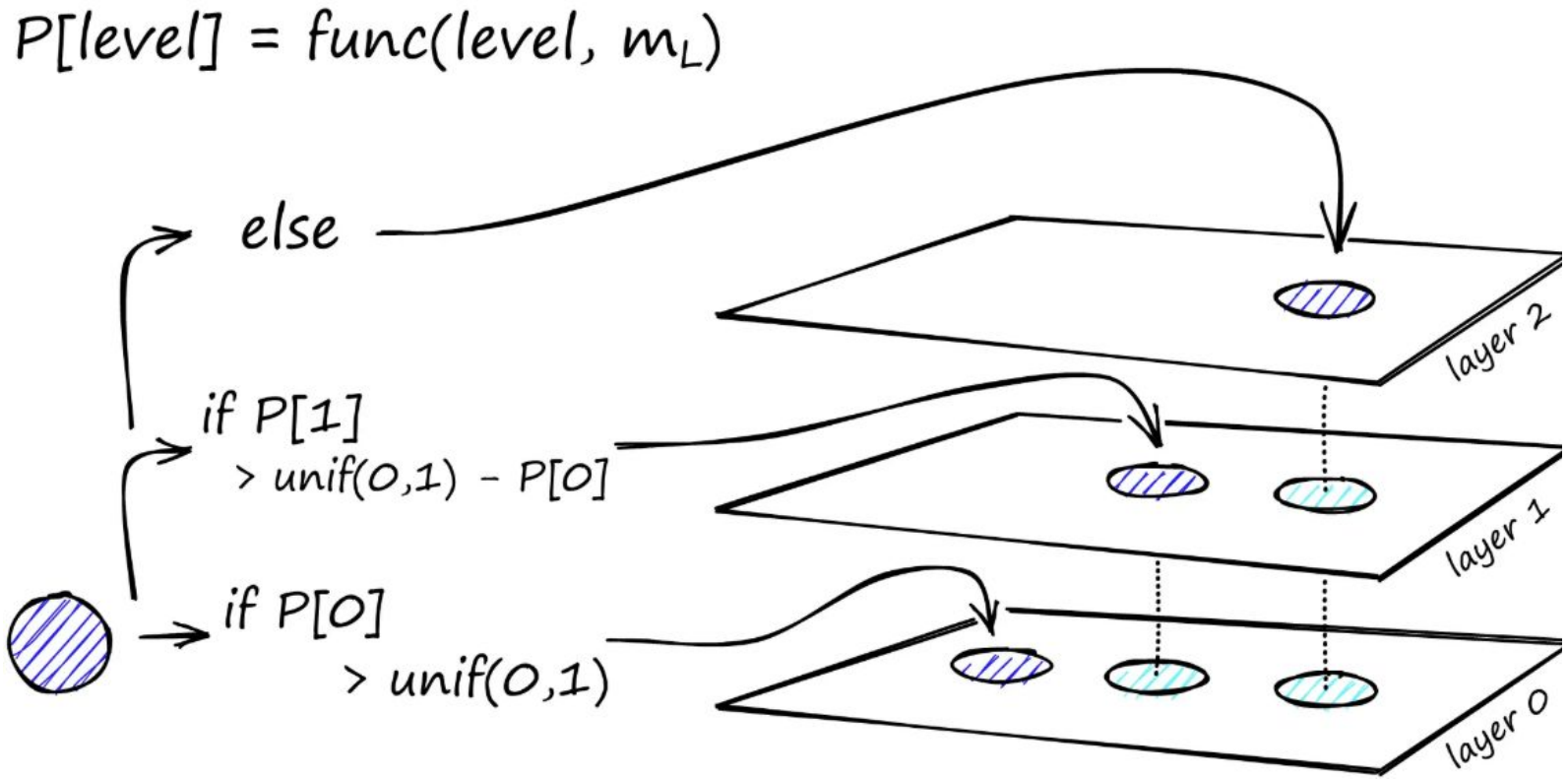
Graph Construction:

Each new vector embedding is assigned a random level (layer) using a geometric probability distribution, ensuring sparse upper layers and dense base layers (layer 0).

M : Maximum number of established connections per node per layer.

eConstruction: Size of the dynamic candidate list used during insertion to find neighbors. Higher values increase accuracy but slow down construction.

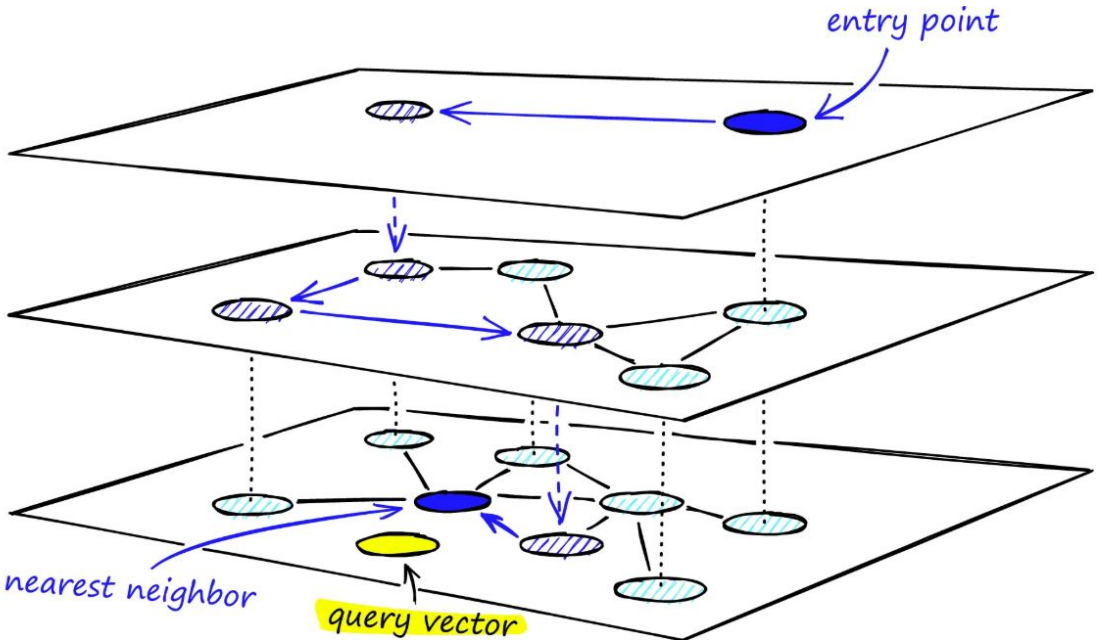
Level Multiplier: Normalizes the probability of a vector being added to higher layers.



Probability for each layer is calculated and used for insertion of a new vector. Layer0 contains all the vectors and the number decreases as we move up.

HNSW Implementation

Just like Skip lists, start with a random entry point, use the NSW algorithm and reach bottom layer when NSW condition is reached.



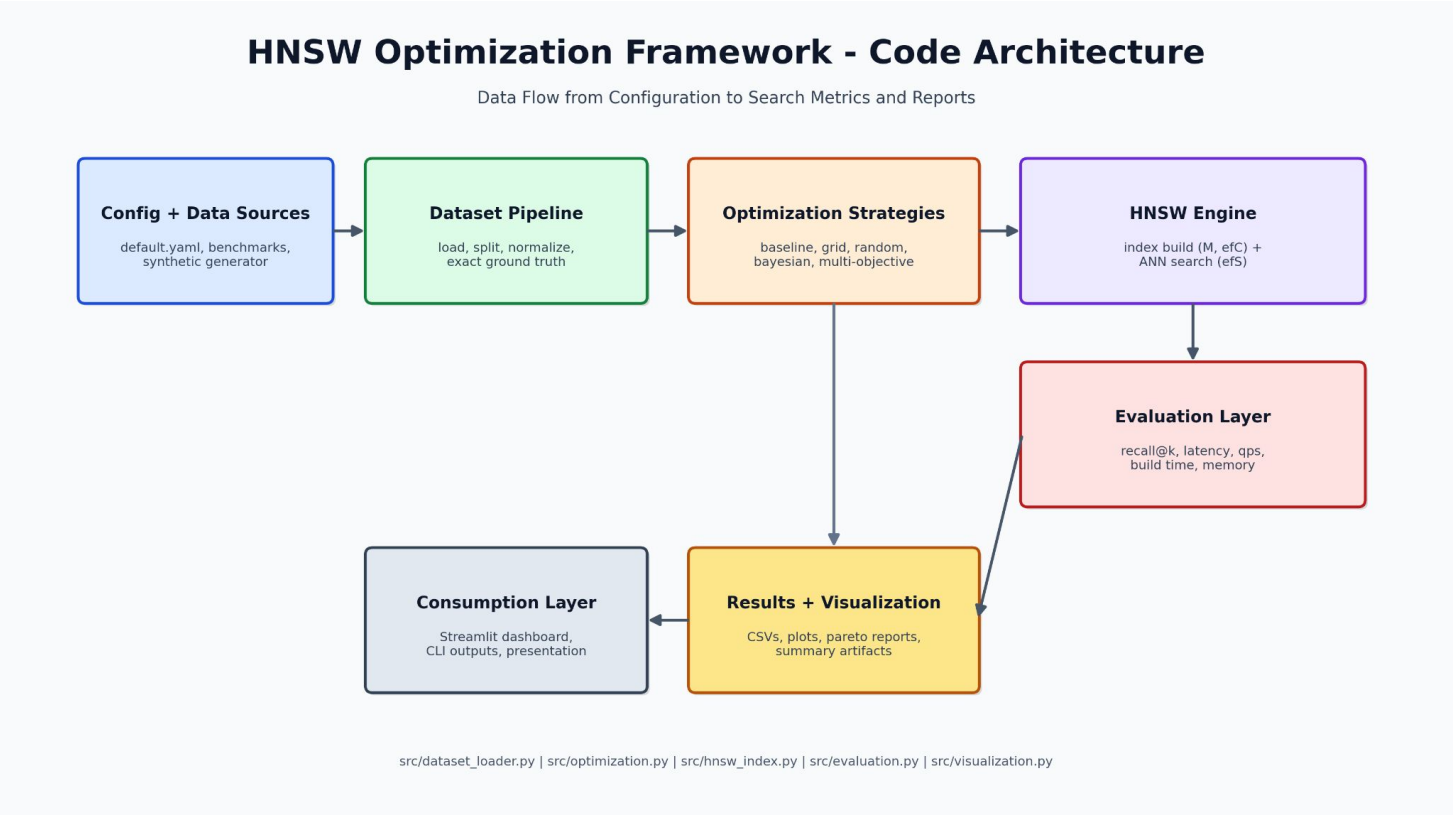
Performance metrics

- **Latency_p95_ms**: A tail-latency metric where 95% of queries complete within this time (in ms) where slowest 5% take longer. Higher **efSearch** → better recall but higher latency.

- **Recall@10**: Fraction of true top-10 nearest neighbors found in the predicted top-10. Measures search quality.

- **Build time (s)**: Time to construct the HNSW index. Higher recall settings often increase build time.
- **Objective is to maximise the Score** = `recall@10 + 0.1 * min(QPS/1000, 1) # indicates latency`

Code structure



Thank you!